

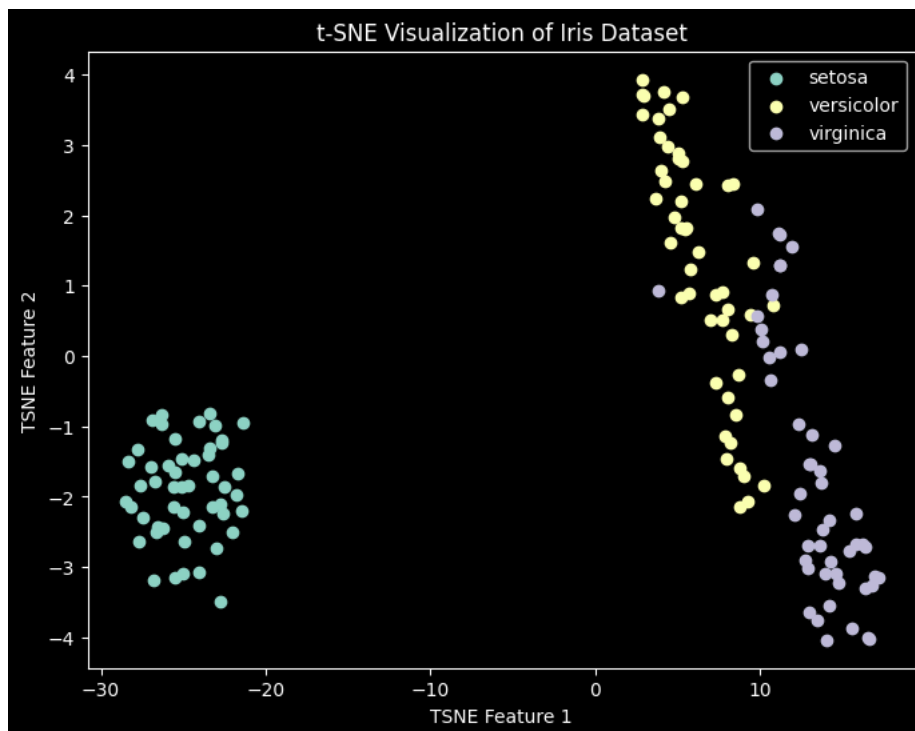
```
''' Name : Amit Kumar, Regd No. : 2341011129, Sl No. : 22 '''
```

```
# Q1
from sklearn.datasets import load_iris
from sklearn.manifold import TSNE
import matplotlib.pyplot as plt

iris = load_iris()
X = iris.data
y = iris.target
tsne = TSNE(n_components=2, random_state=42)
X_tsne = tsne.fit_transform(X)
plt.figure(figsize=(8,6))

for i in range(3):
    plt.scatter(
        X_tsne[y == i, 0],
        X_tsne[y == i, 1],
        label=iris.target_names[i]
    )

plt.xlabel("TSNE Feature 1")
plt.ylabel("TSNE Feature 2")
plt.title("t-SNE Visualization of Iris Dataset")
plt.legend()
plt.show()
```



```
''' Name : Amit Kumar, Regd No. : 2341011129, Sl No. : 22 '''
```

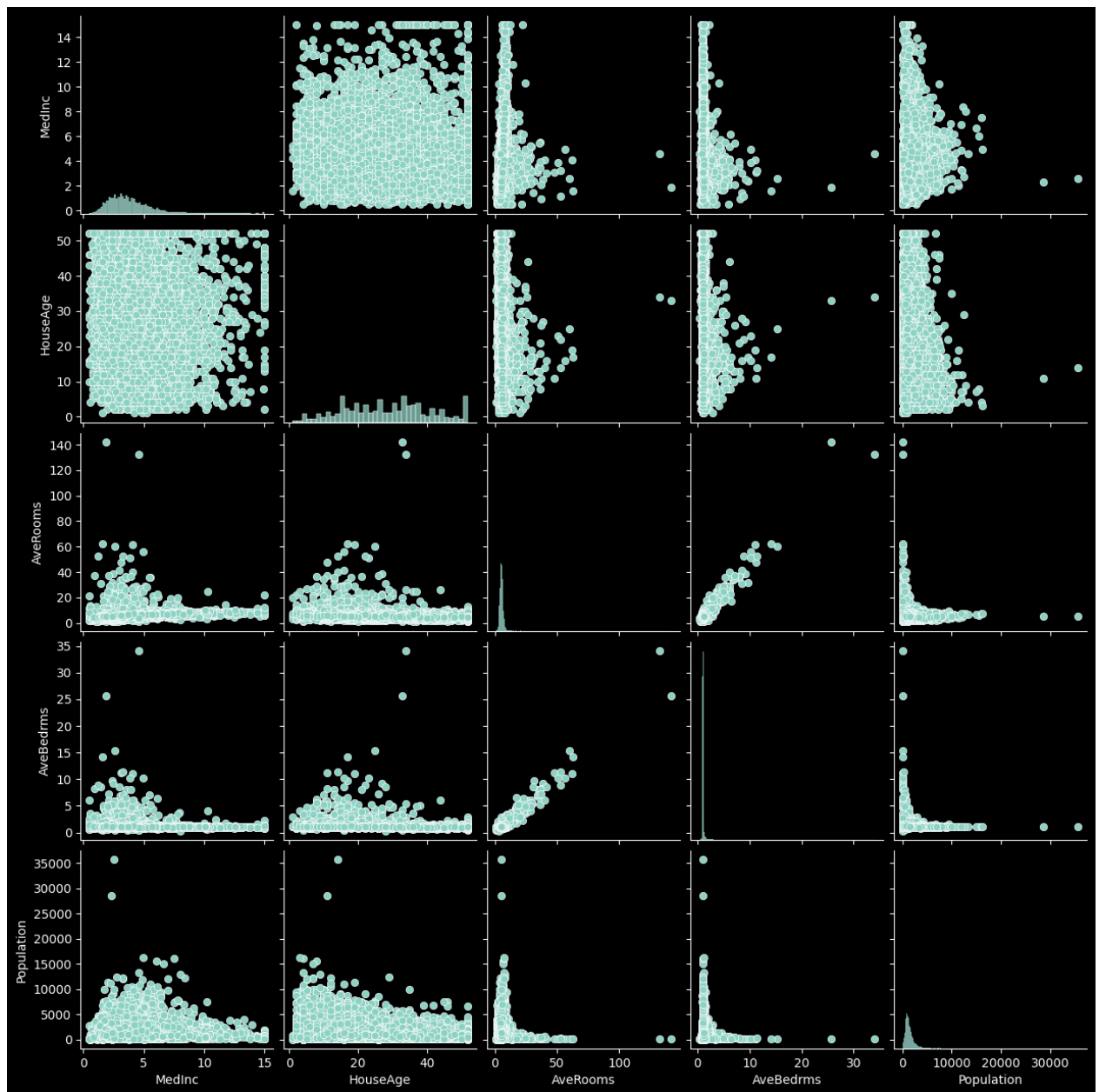
```
# Q2
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.datasets import fetch_california_housing

housing = fetch_california_housing()

df = pd.DataFrame(housing.data, columns=housing.feature_names)

sns.pairplot(df.iloc[:, :5])

plt.show()
```



''' Name : Amit Kumar, Regd No. : 2341011129, Sl No. : 22 '''

```
# Q3
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
import numpy as np

years = np.array([2000, 2005, 2010, 2015, 2020]).reshape(-1,1)
temps = np.array([55.1, 55.5, 56.0, 56.4, 57.0])

model = LinearRegression()

model.fit(years, temps)

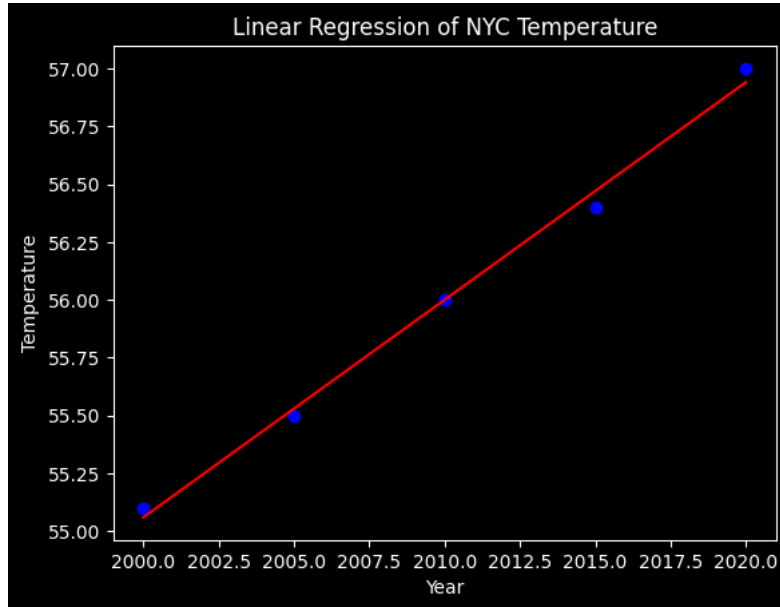
predicted = model.predict(years)
```

```
plt.scatter(years, temps, color='blue')
plt.plot(years, predicted, color='red')

plt.xlabel("Year")
plt.ylabel("Temperature")
plt.title("Linear Regression of NYC Temperature")

plt.show()

print("Slope:", model.coef_[0])
print("Intercept:", model.intercept_)
```



```
Slope: 0.09399999999999994
Intercept: -132.93999999999988
```

```
''' Name : Amit Kumar, Regd No. : 2341011129, Sl No. : 22 '''
```

```
# Q4
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score

iris = load_iris()
X = iris.data
y = iris.target
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

knn = KNeighborsClassifier()
knn.fit(X_train, y_train)
y_pred = knn.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
print("Prediction Accuracy:", accuracy * 100)
```

```
Prediction Accuracy: 100.0
```

```
''' Name : Amit Kumar, Regd No. : 2341011129, Sl No. : 22 '''
```

```
# Q5
import math

points = {
    'A': {'x': 2.0, 'y': 3.0, 'class': 0},
    'B': {'x': 1.0, 'y': 1.0, 'class': 0},
    'C': {'x': 4.0, 'y': 4.0, 'class': 1},
    'D': {'x': 5.0, 'y': 2.0, 'class': 1}
}

px, py = 3.0, 3.0
distances = []
for point, values in points.items():
    distance = math.sqrt(
        (px - values['x'])**2 +
        (py - values['y'])**2
    )
```

```

    )
    distances.append((point, distance, values['class']))

distances.sort(key=lambda x: x[1])
print("Distances from Point P(3,3):\n")

for point, distance, cls in distances:

    print(
        f"Point {point} --> Distance = {distance:.2f}, Class = {cls}"
    )
k = 3
nearest_neighbors = distances[:k]
class_count = {}

for neighbor in nearest_neighbors:
    cls = neighbor[2]
    class_count[cls] = class_count.get(cls, 0) + 1

predicted_class = max(class_count, key=class_count.get)

print("\nNearest Neighbors:")

for n in nearest_neighbors:
    print(n)
print(f"\nPredicted Class for Point P(3,3) = {predicted_class}")

```

Distances from Point P(3,3):

```

Point A --> Distance = 1.00, Class = 0
Point C --> Distance = 1.41, Class = 1
Point D --> Distance = 2.24, Class = 1
Point B --> Distance = 2.83, Class = 0

```

Nearest Neighbors:

```

('A', 1.0, 0)
('C', 1.4142135623730951, 1)
('D', 2.23606797749979, 1)

```

Predicted Class for Point P(3,3) = 1

```
''' Name : Amit Kumar, Regd No. : 2341011129, Sl No. : 22 '''
```

```

# Q6
import math

students = [
    ([85, 90, 88], "Pass"),
    ([70, 75, 80], "Pass"),
    ([60, 65, 70], "Fail"),
    ([50, 55, 58], "Fail"),
    ([95, 92, 96], "Pass"),
    ([45, 50, 48], "Fail")
]
new_student = [72, 78, 75]

distances = []
for data, label in students:
    distance = math.sqrt(
        (new_student[0] - data[0])**2 +
        (new_student[1] - data[1])**2 +
        (new_student[2] - data[2])**2
    )
    distances.append((distance, label))
distances.sort(key=lambda x: x[0])

print("Distances from New Student:\n")

for d in distances:
    print(f"Distance = {d[0]:.2f}, Class = {d[1]}")

k = 3
nearest_neighbors = distances[:k]

label_count = {}
for neighbor in nearest_neighbors:
    label = neighbor[1]
    label_count[label] = label_count.get(label, 0) + 1

prediction = max(label_count, key=label_count.get)

print("\nNearest Neighbors:")
for n in nearest_neighbors:

```

```
print(n)
print(f"\nPredicted Result for Student = {prediction}")
```

Distances from New Student:

```
Distance = 6.16, Class = Pass
Distance = 18.38, Class = Fail
Distance = 21.95, Class = Pass
Distance = 34.15, Class = Pass
Distance = 36.08, Class = Fail
Distance = 47.35, Class = Fail
```

Nearest Neighbors:

```
(6.164414002968976, 'Pass')
(18.384776310850235, 'Fail')
(21.95449840010015, 'Pass')
```

Predicted Result for Student = Pass

```
''' Name : Amit Kumar, Regd No. : 2341011129, Sl No. : 22 '''
```

Q7

```
from sklearn.datasets import load_iris
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import cross_val_score
import matplotlib.pyplot as plt
```

```
iris = load_iris()
X = iris.data
y = iris.target
```

```
k_values = range(1, 21)
scores = []
```

```
for k in k_values:
    knn = KNeighborsClassifier(n_neighbors=k)
    score = cross_val_score(knn, X, y, cv=5)
    scores.append(score.mean())
```

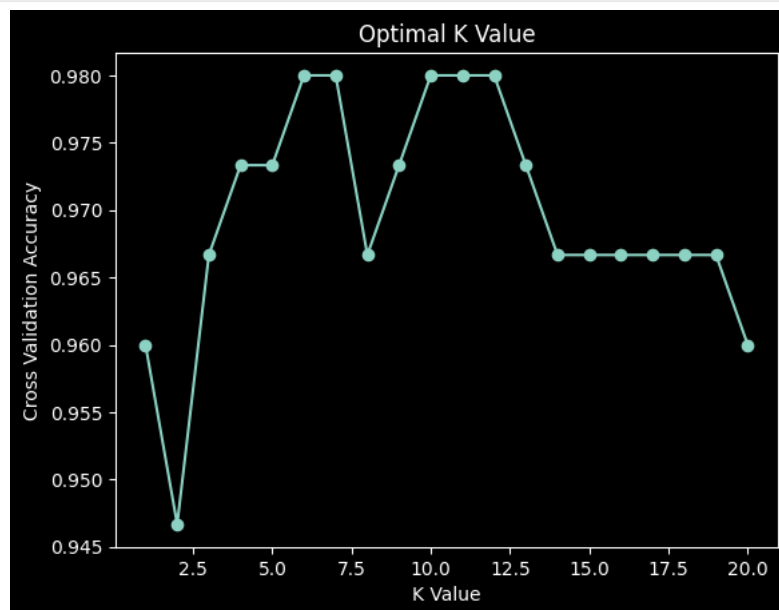
```
plt.plot(k_values, scores, marker='o')
```

```
plt.xlabel("K Value")
plt.ylabel("Cross Validation Accuracy")
plt.title("Optimal K Value")
```

```
plt.show()
```

```
best_k = k_values[scores.index(max(scores))]
```

```
print("Best K:", best_k)
```



Best K: 6

```
''' Name : Amit Kumar, Regd No. : 2341011129, Sl No. : 22 '''
```

Q8

```
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
import numpy as np
```

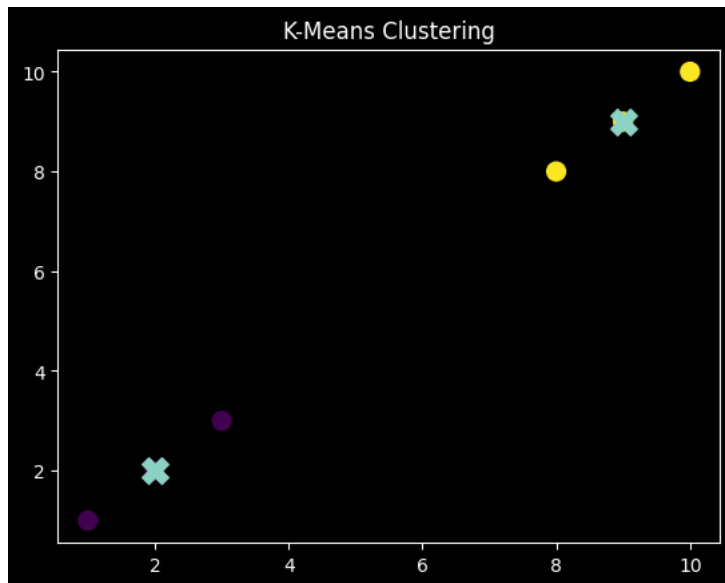
```

X = np.array([
    [1,1],
    [2,2],
    [3,3],
    [8,8],
    [9,9],
    [10,10]
])

kmeans = KMeans(n_clusters=2, random_state=42)
kmeans.fit(X)
plt.scatter(X[:,0], X[:,1], c=kmeans.labels_, s=100)
plt.scatter(
    kmeans.cluster_centers_[0,0],
    kmeans.cluster_centers_[0,1],
    marker='x',
    s=200
)

plt.title("K-Means Clustering")
plt.show()

```



```

''' Name : Amit Kumar, Regd No. : 2341011129, Sl No. : 22 '''

# Q9
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans

df = pd.read_csv("Mall_Customers.csv")
X = df[['Annual Income (k$)', 'Spending Score (1-100)']]

wcss = []

for i in range(1,11):
    kmeans = KMeans(n_clusters=i, random_state=42)
    kmeans.fit(X)
    wcss.append(kmeans.inertia_)

plt.plot(range(1,11), wcss, marker='o')

plt.xlabel("Number of Clusters")
plt.ylabel("WCSS")
plt.title("Elbow Method")
plt.show()

kmeans = KMeans(n_clusters=5, random_state=42)
y_kmeans = kmeans.fit_predict(X)
plt.scatter(X.iloc[:,0], X.iloc[:,1], c=y_kmeans)

plt.xlabel("Annual Income")
plt.ylabel("Spending Score")
plt.title("Customer Segments")

plt.show()

```

```
''' Name : Amit Kumar, Regd No. : 2341011129, Sl No. : 22 '''
```

```
# Q10
```

```
#a
```

```
import pandas as pd
s = pd.Series([7,11,13,17])
print(s,"\n")
```

```
#b
```

```
import pandas as pd
s = pd.Series([100.0] * 5)
print(s,"\n")
```

```
#c
```

```
import pandas as pd
import numpy as np
s = pd.Series(np.random.randint(0,101,20))
print(s)
print(s.describe(),"\n")
```

```
#d
```

```
import pandas as pd
temperatures = pd.Series(
    [98.6, 98.9, 100.2, 97.9],
    index=['Julie', 'Charlie', 'Sam', 'Andrea']
)
print(temperatures,"\n")
```

```
#e
```

```
import pandas as pd
temp_dict = {
    'Julie': 98.6,
    'Charlie': 98.9,
    'Sam': 100.2,
    'Andrea': 97.9
}
temperatures = pd.Series(temp_dict)
print(temperatures,"\n")
```

```
0    7
1   11
2   13
3   17
dtype: int64
```

```
0    100.0
1    100.0
2    100.0
3    100.0
4    100.0
dtype: float64
```

```
0     83
1     88
2     16
3     77
4     19
5     20
6      5
7     73
8     13
9     83
10    36
11    71
12    61
13    15
14    65
15    87
16    23
17    14
18    93
19    27
dtype: int32
count    20.00000
mean     48.45000
std      31.80777
min       5.00000
25%      18.25000
50%      48.50000
75%      78.50000
max      93.00000
dtype: float64
```

```
Julie      98.6
Charlie    98.9
```

```
Sam      100.2
Andrea   97.9
dtype: float64
```

```
Julie     98.6
Charlie   98.9
Sam       100.2
Andrea    97.9
dtype: float64
```